



Department of Cyber Security

BS Cyber Security

NASTP Institute of Information Technology

Submitted By:

Muhammad Haseeb Sajid (F2024CY018)

Murtajiz Hussain Naqvi (F2024CY042)

Ayesha Khan (F2024CY039)

Submitted To: Ma'am Sibgha

Semester: IV

Session: Fall – 2024 to 2028

Subject: COAL Lab

Table of Contents

1. Objective.....	3
2. Theory.....	3
Introduction to Password Managers.....	3
Assembly Language.....	3
Irvine32 Library.....	4
Encryption Techniques Used.....	4
1. XOR Encryption.....	4
2. Caesar Cipher.....	5
Procedures.....	5
Arrays and Memory Handling.....	6
Stack Operations.....	6
Checksum Validation.....	6
3. Lab Activity.....	6
Program Initialization.....	6
Master PIN Authentication.....	7
Add Password Feature.....	7
View Passwords.....	7
Search Password by Website.....	8
Encrypt / Decrypt Text.....	8
Strong Password Generator.....	8
Testing and Debugging.....	8
4. Screenshots.....	9
Main Menu.....	9
PIN Authentication.....	9
Add Password Output.....	10
View Password Output.....	10
Search Feature Output.....	11
Encryption / Decryption Output.....	11
Password Generator Output.....	12
5. Conclusion.....	12

1. Objective

The objective of this project is to design and implement a secure Password Manager using x86 Assembly Language with the Irvine32 library. The project demonstrates practical implementation of low-level programming concepts including arrays, procedures, loops, encryption, memory handling, and modular programming.

The system allows users to:

- Store passwords securely
- Search saved credentials
- Encrypt and decrypt text
- Generate strong passwords
- Protect access using a Master PIN

This project also aims to strengthen understanding of Computer Organization and Assembly Language concepts through real-world application development.

2. Theory

Introduction to Password Managers

A password manager is a software application that stores and manages user credentials securely. Since users often have many online accounts, password managers help organize and protect sensitive information.

In this project, SecureVault is implemented entirely in Assembly Language to demonstrate how security systems can be developed at low level using processor instructions and memory operations.

Assembly Language

Assembly Language is a low-level programming language that communicates directly with the processor architecture. It provides:

- Direct memory access
- Register manipulation
- Fast execution
- Efficient hardware-level control

The project uses:

- MASM (Microsoft Macro Assembler)
- Irvine32 Library

Encryption Techniques Used

1. XOR Encryption

XOR encryption is a simple symmetric encryption method. Each character is XORed with a key value.

Formula:

Encrypted Character = Character XOR Key

In this project:

- XOR key = ABh

Advantages:

- Fast execution
- Easy implementation in Assembly
- Reversible using same operation

2. Caesar Cipher

Caesar Cipher shifts alphabetic characters by a fixed number.

Example:

- $A \rightarrow D$
- $B \rightarrow E$

In this project:

- Shift value = 3

Encryption Formula:

$$Encrypted = Original + Shift$$

Decryption Formula:

$$Original = Encrypted - Shift$$

Procedures

The program is divided into modular procedures such as:

- AddPassword
- ViewPasswords
- SearchPassword
- EncryptMenu
- GeneratePassword
- XorCrypt
- CaesarEncrypt
- CaesarDecrypt

This improves:

- Code readability
- Reusability
- Structured programming

Arrays and Memory Handling

Passwords, usernames, and websites are stored using arrays.

Example:

- websites array
- usernames array
- passwords array

Each entry occupies fixed memory space using FIELD_LEN.

Stack Operations

The project uses:

- PUSH
- POP
- PUSHA
- POPA

These instructions preserve register values during procedure calls.

Checksum Validation

A checksum is calculated for passwords to ensure data integrity.

The checksum:

- Adds ASCII values of characters
- Detects possible corruption or modification

3. Lab Activity

Program Initialization

- Included Irvine32 library
- Defined constants and data section
- Created menu interface

Master PIN Authentication

A Master PIN system was implemented to:

- Restrict unauthorized access
- Limit login attempts
- Lock the vault after multiple failures

Features:

- Hidden PIN input using *
- Attempt counter
- Access validation

Add Password Feature

The user enters:

- Website name
- Username
- Password

The password is then:

1. XOR encrypted
2. Caesar encrypted
3. Stored in memory

View Passwords

Stored passwords are:

- Retrieved from arrays
- Decrypted temporarily
- Displayed to the user

This required:

- Memory copying
- Decryption procedures
- String handling

Search Password by Website

The search module:

- Takes website input
- Compares strings using StrCompare
- Displays matching credentials

Encrypt / Decrypt Text

Users can manually:

- Encrypt text
- Decrypt encrypted text

This demonstrates:

- XOR encryption
- Caesar Cipher implementation

Strong Password Generator

The program generates random passwords using:

- Uppercase letters
- Lowercase letters
- Digits
- Special symbols

Random values are generated using:

- Randomize
- RandomRange

Testing and Debugging

The project was tested for:

- Invalid inputs
- Maximum entries
- Encryption/decryption accuracy
- Search functionality
- PIN validation

Errors were debugged using:

- Register tracking
- Procedure testing
- Memory inspection

4. Screenshots

Main Menu

```
C:\> D:\Homework\COAL Lab\Repo\Project2\Debug\Project2.exe

=====
SecureVault - Password Manager
NASTP Institute - COAL Project
=====

1. Add New Password
2. View All Passwords
3. Search Password by Website
4. Encrypt / Decrypt Text
5. Generate Strong Password
6. Exit

Enter choice:
```

PIN Authentication

```
C:\> D:\Homework\COAL Lab\Repo\Project2\Debug\Project2.exe

=====
SecureVault - Password Manager
NASTP Institute - COAL Project
=====

Enter Master PIN:
```

Add Password Output

 D:\Homework\COAL Lab\Repo\Project2\Debug\Project2.exe

```
=====
SecureVault - Password Manager
NASTP Institute - COAL Project
=====
```

1. Add New Password
2. View All Passwords
3. Search Password by Website
4. Encrypt / Decrypt Text
5. Generate Strong Password
6. Exit

Enter choice:

Website : Instagram

Username : Haseeb

Password : *****

[+] Password saved successfully!

Press any key to continue...

View Password Output

```
=====
SecureVault - Password Manager
NASTP Institute - COAL Project
=====
```

1. Add New Password
2. View All Passwords
3. Search Password by Website
4. Encrypt / Decrypt Text
5. Generate Strong Password
6. Exit

Enter choice:

Website : Instagram

Username : Haseeb

Password : Hackzz@31

Website : Facebook

Username : Hussain

Password : boook63

Press any key to continue...

Search Feature Output

```
-----  
Website : Instagram  
Username : Haseeb  
Password : Hackzz@31  
  
Press any key to continue...
```

Encryption / Decryption Output

```
=====
SecureVault - Password Manager
NASTP Institute - COAL Project
=====

1. Add New Password
2. View All Passwords
3. Search Password by Website
4. Encrypt / Decrypt Text
5. Generate Strong Password
6. Exit

Enter choice:
1-Encrypt 2-Decrypt :
Enter text to encrypt: Secrets
Encrypted : 0x11111111

Press any key to continue...
```

```
=====
SecureVault - Password Manager
NASTP Institute - COAL Project
=====

1. Add New Password
2. View All Passwords
3. Search Password by Website
4. Encrypt / Decrypt Text
5. Generate Strong Password
6. Exit

Enter choice:
1-Encrypt 2-Decrypt :
Enter text to decrypt: 0x11111111
Decrypted : Secrets

Press any key to continue...
```

Password Generator Output

```
=====
SecureVault - Password Manager
NASTP Institute - COAL Project
=====

1. Add New Password
2. View All Passwords
3. Search Password by Website
4. Encrypt / Decrypt Text
5. Generate Strong Password
6. Exit

Enter choice:
Enter desired length (8-20): 18
Generating strong password...
Generated : JLD$?%Aj08&82@TP?b

Press any key to continue...
```

5. Conclusion

The SecureVault project successfully demonstrated the practical implementation of Assembly Language concepts through a real-world security application.

The project covered important COAL topics including:

- Character input/output
- Arrays and strings
- Procedures
- Loops and conditional execution
- Stack operations
- Encryption techniques
- Memory management

By developing this project, we gained hands-on experience with low-level programming, debugging, modular design, and secure data handling. The project also improved our understanding of how encryption and authentication systems work internally at processor level. Overall, SecureVault effectively bridges theoretical COAL concepts with practical cybersecurity applications.